

UP LT GRADE • COMPUTER SCIENCE MAINS

DBMS — High-Probability Numerical & Analytical Questions with Full Solutions

Selected for Likelihood of Appearing, Based on the Confirmed Exam Pattern

Why these 10: DBMS Mains questions are rarely pure arithmetic — they are analytical/derivation numerals (closures, decompositions, query tracing, serializability checks) that behave exactly like the compound, multi-part questions seen in the conducted Science paper.

Each question is tagged with its appearance likelihood based on how consistently it's tested across DBMS courses at this level.

Practice set • step-by-step solutions • exam-style presentation

Table of Contents

Q1. Attribute Closure Computation (X^+)	Page 3
Q2. Finding Candidate Keys from FDs	Page 4
Q3. Normalization — Decompose to 2NF	Page 5
Q4. Normalization — Decompose to 3NF	Page 6
Q5. Relational Algebra Expression Evaluation	Page 7
Q6. SQL Query Output Tracing (GROUP BY / HAVING)	Page 8
Q7. ER-to-Relational Schema Mapping	Page 9
Q8. Conflict Serializability — Precedence Graph	Page 10
Q9. Lossless Join Decomposition Test	Page 11
Q10. Lost Update Anomaly — Schedule Trace	Page 12

Q1. Attribute Closure Computation

Very High Chance

Question

Given relation $R(A, B, C, D, E)$ with Functional Dependencies: $A \rightarrow B$, $BC \rightarrow D$, $D \rightarrow E$. Compute the closure of $\{A, C\}$, i.e. find $\{A, C\}^+$.

Step-by-Step Closure Computation

Start: $X = \{A, C\}$

Apply $A \rightarrow B$: $A \in X$, so add $B \rightarrow X = \{A, B, C\}$

Apply $BC \rightarrow D$: both B and $C \in X$, so add $D \rightarrow X = \{A, B, C, D\}$

Apply $D \rightarrow E$: $D \in X$, so add $E \rightarrow X = \{A, B, C, D, E\}$

No more FDs apply (all attributes already included) — STOP.

Final Answer

$\{A, C\}^+ = \{A, B, C, D, E\}$ = all attributes of R . Since $\{A, C\}$ determines every attribute, **$\{A, C\}$ is a Super Key** of R (and a Candidate Key if no smaller subset also has this property).

Why This Is High-Probability

Attribute closure is the single most foundational DBMS numerical — it's the direct prerequisite for candidate key finding (Q2), normalization (Q3-Q4), and BCNF/lossless-join checks (Q9). Mastering this one technique unlocks 4 other question types.

Q2. Finding Candidate Keys from FDs

Very High Chance

Question

Given relation $R(A, B, C, D, E)$ with Functional Dependencies: $A \rightarrow BC$, $C \rightarrow D$, $D \rightarrow E$. Find all candidate keys of R .

Step 1: Compute Closure of A

$X = \{A\}$
 $A \rightarrow BC$: add B, $C \rightarrow X = \{A, B, C\}$
 $C \rightarrow D$: add D $\rightarrow X = \{A, B, C, D\}$
 $D \rightarrow E$: add E $\rightarrow X = \{A, B, C, D, E\}$

Step 2: Check Minimality

$A^+ = \{A, B, C, D, E\} = \text{ALL attributes}$, so A is a Super Key.
Since A is a SINGLE attribute, it cannot be reduced further — it is automatically minimal.

Final Answer

A is the only Candidate Key of R (attributes B, C, D, E never appear on the left side of any FD that could lead back to A, so no other attribute or combination can be a key).

Why This Is High-Probability

"Find all candidate keys" is one of the most repeated standalone DBMS Mains questions — examiners test whether you can systematically compute closures for candidate attributes rather than guessing.

Q3. Normalization — Decompose to 2NF

Very High Chance

Question

Table Enrollment(StudentID, CourseID, StudentName, CourseName, Grade) has Primary Key (StudentID, CourseID), with FDs: StudentID → StudentName, CourseID → CourseName, (StudentID, CourseID) → Grade. Decompose this table into 2NF.

Step 1: Identify Partial Dependencies

Primary key is composite: (StudentID, CourseID).

StudentID → StudentName: depends on ONLY PART of the key (StudentID alone) → **Partial Dependency**

CourseID → CourseName: depends on ONLY PART of the key (CourseID alone) → **Partial Dependency**

(StudentID, CourseID) → Grade: depends on the WHOLE key → OK, no violation

Step 2: Decompose to Remove Partial Dependencies

Move each partially-dependent attribute (with its determinant) into its own table:

Final Answer — 2NF Decomposition

Student(StudentID, StudentName)

Course(CourseID, CourseName)

Enrollment(StudentID, CourseID, Grade)

Why This Is High-Probability

2NF questions ALWAYS use a composite-key table like this one, since 2NF is meaningless for single-attribute keys — this exact "Student-Course-Grade" style pattern is the textbook-standard example examiners draw from.

Q4. Normalization — Decompose to 3NF

Very High Chance

Question

Table Employee(EmpID, EmpName, DeptID, DeptName) has Primary Key EmpID, with FDs: EmpID → EmpName, EmpID → DeptID, DeptID → DeptName. Decompose this table into 3NF.

Step 1: Identify Transitive Dependency

EmpID → DeptID (key determines DeptID directly) DeptID → DeptName (non-key determines another non-key)
This means: EmpID → DeptID → DeptName, so EmpID determines DeptName only TRANSITIVELY (through DeptID) — this is a **Transitive Dependency**, violating 3NF.

Step 2: Decompose to Remove the Transitive Dependency

Split off the transitively-dependent attribute (DeptName) into a new table keyed by its true determinant (DeptID):

Final Answer — 3NF Decomposition

Employee(EmpID, EmpName, DeptID)

Department(DeptID, DeptName)

DeptID in Employee is now a Foreign Key referencing Department.

Why This Is High-Probability

Transitive dependency (non-key → non-key) is the SPECIFIC pattern 3NF questions test, as opposed to 2NF's partial dependency (part-of-key → non-key) — examiners frequently ask both back-to-back to check you know the difference.

Q5. Relational Algebra Expression Evaluation

High Chance

Question

Given the tables below, evaluate: $\pi_{\text{Name}} (\sigma_{\text{Dept}=\text{'CS'}}(\text{Student}) \bowtie \text{Enroll})$

Given Data

Student			Enroll	
RollNo	Name	Dept	RollNo	CourseID
1	Amit	CS	1	C101
2	Neha	EC	3	C102
3	Ravi	CS	4	C103
4	Sita	ME		

Step 1: Apply Selection $\sigma_{\text{Dept}=\text{'CS'}}(\text{Student})$

Result: $\{(1, \text{Amit}, \text{CS}), (3, \text{Ravi}, \text{CS})\}$

Step 2: Natural Join with Enroll (on RollNo)

RollNo=1 matches Enroll(1,C101) $\rightarrow (1, \text{Amit}, \text{CS}, \text{C101})$
RollNo=3 matches Enroll(3,C102) $\rightarrow (3, \text{Ravi}, \text{CS}, \text{C102})$
(RollNo=4, Sita, was filtered out in Step 1 since Dept $\neq$ CS)

Step 3: Apply Projection π_{Name}

Keep only the Name column from the joined result.

Final Answer

Result = **{Amit, Ravi}** — the names of CS department students who are enrolled in at least one course.

Q6. SQL Query Output Tracing

High Chance

Question

For the Employee table below, trace the output of:

```
SELECT DeptID, COUNT(*), AVG(Salary) FROM Employee GROUP BY DeptID HAVING COUNT(*) > 1;
```

Given Data

EmpID	Name	Salary	DeptID
1	A	50000	10
2	B	60000	10
3	C	55000	20
4	D	45000	30
5	E	65000	30
6	F	70000	30

Step 1: Group by DeptID

Dept 10: {50000, 60000} → count=2, avg=55000
Dept 20: {55000} → count=1, avg=55000
Dept 30: {45000, 65000, 70000} → count=3, avg=60000

Step 2: Apply HAVING COUNT(*) > 1 (removes Dept 20)

Final Answer — Query Output

DeptID	COUNT(*)	AVG(Salary)
10	2	55000
30	3	60000

Dept 20 is excluded since it has only 1 employee (fails the HAVING COUNT(*) > 1 condition).

Q7. ER-to-Relational Schema Mapping

High Chance

Question

An ER diagram has: Entity Student (key: RollNo, attribute: Name), Entity Course (key: CourseID, attribute: Title), and a Many-to-Many relationship "Enrolls" between them carrying attribute Grade. Map this ER design to relational schema (tables).

General Mapping Rules (apply based on cardinality)

Relationship Type	Mapping Rule
1:1	Add either entity's key as a Foreign Key in the other table (or merge tables)
1:N	Add the "1" side's key as a Foreign Key in the "N" side's table
M:N	Create a SEPARATE table for the relationship; its Primary Key is the COMBINATION of both entities' keys

Applying the M:N Rule to This Case

Since "Enrolls" is Many-to-Many, it becomes its own table, keyed by BOTH RollNo and CourseID together (their combination uniquely identifies one enrollment record), also holding the relationship's own attribute (Grade).

Final Answer — Relational Schema

Student(RollNo, Name)

Course(CourseID, Title)

Enrolls(RollNo, CourseID, Grade) — RollNo references Student, CourseID references Course

Q8. Conflict Serializability — Precedence Graph

High Chance

Question

For the schedule S: $R_1(A)$, $R_2(A)$, $W_1(A)$, $W_2(A)$ — determine whether S is conflict serializable using a precedence graph.

Step 1: Identify All Conflicting Operation Pairs

A conflict exists between two operations on the SAME data item, from DIFFERENT transactions, where at least ONE is a Write.

$R_1(A)$ before $W_2(A)$ → conflict → edge $T1 \rightarrow T2$

$R_2(A)$ before $W_1(A)$ → conflict → edge $T2 \rightarrow T1$

$W_1(A)$ before $W_2(A)$ → conflict → edge $T1 \rightarrow T2$

Step 2: Draw the Precedence Graph

Nodes: T1, T2

Edges: $T1 \rightarrow T2$ (from $R1-W2$ and $W1-W2$), $T2 \rightarrow T1$ (from $R2-W1$)

This graph contains edges in BOTH directions between T1 and T2 — a **cycle** ($T1 \rightarrow T2 \rightarrow T1$).

Final Answer

The precedence graph HAS a cycle, so schedule S is **NOT conflict serializable**.

Rule: A schedule is conflict serializable if and only if its precedence graph is ACYCLIC.

Q9. Lossless Join Decomposition Test

Moderate Chance

Question

Relation $R(A, B, C)$ is decomposed into $R_1(A, B)$ and $R_2(B, C)$, given the FD $B \rightarrow C$ holds on R . Verify whether this decomposition is Lossless Join.

Step 1: Apply the Lossless Join Test Formula

Formula

A decomposition into R_1 and R_2 is lossless if and only if:

$(R_1 \cap R_2) \rightarrow R_1$ OR $(R_1 \cap R_2) \rightarrow R_2$ (using the given FDs)

Step 2: Compute and Check

$R_1 \cap R_2 = \{B\}$ (the common attribute)

Check if $B \rightarrow R_2$: $R_2 = \{B, C\}$. Since $B \rightarrow C$ is GIVEN, and $B \rightarrow B$ trivially holds, we get $B \rightarrow BC$, i.e., **$B \rightarrow R_2$ holds.**

Final Answer

Since $(R_1 \cap R_2) = \{B\}$ functionally determines R_2 entirely, the decomposition **IS Lossless Join.**

Q10. Lost Update Anomaly — Schedule Trace

Moderate Chance

Question

Account balance A starts at 1000. Two transactions run concurrently without locking:

T1: Read(A); A = A+100; Write(A)

T2: Read(A); A = A+200; Write(A)

If both transactions read A BEFORE either writes, trace the final value of A and identify the anomaly.

Step 1: Trace the Interleaved Execution

Time	T1	T2	Value of A
t1	Read(A) → 1000		1000
t2		Read(A) → 1000	1000
t3	A = 1000+100 = 1100; Write(A)		1100
t4		A = 1000+200 = 1200; Write(A)	1200

Step 2: Compare with the Expected Correct Result

Correct result if run serially (one after another) should be: $1000 + 100 + 200 = 1300$

But the actual final value after this interleaved schedule is only **1200** — T1's +100 update was completely overwritten and lost.

Final Answer

Final value of A = 1200 (incorrect). This is the **Lost Update Problem** — T2 read A before T1's write was visible, so T2's write silently overwrote T1's update. Prevented using proper locking (Exclusive locks) or Two-Phase Locking (2PL).

— End of Practice Set —

DBMS Numerical & Analytical Questions | UP LT Grade CS Mains Preparation